Chat Scholar — Flask + Groq + FAISS

Complete installation, terminal execution and testing guide based on your uploaded app.py.

1. What this project uses

Your app is a Flask web application. It reads PDF files with PyPDF2, creates lightweight 384-dimensional local embeddings, stores them in a FAISS vector index, retrieves relevant chunks, and sends the context to a Groq model. It also contains an essay-grading feature using Groq. ■filecite■turn0file0■L10-L14■ ■filecite■turn0file0■L72-L76■ ■filecite■turn0file0■L187-L210■

2. Create the project folder

```
Windows PowerShell
mkdir ChatScholar
cd ChatScholar
code .
```

3. Put your Python file in the folder

Rename the uploaded file app(1).py to app.py and keep it inside the ChatScholar folder. Your code starts by importing NumPy, FAISS, dotenv, Flask, PyPDF2 and Groq. ■filecite■turn0file0■L1-L14■

4. Create and activate a Python 3.12 environment

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1

# Check:
python --version
where python
```

If PowerShell blocks activation

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

5. Upgrade pip

```
python -m pip install --upgrade pip
```

6. Install every required package

```
python -m pip install flask python-dotenv numpy faiss-cpu PyPDF2 groq werkzeug
```

Your code specifically uses Flask, dotenv, PyPDF2, Groq, NumPy and FAISS, so these are the packages to install. ■filecite■turn0file0■L7-L14■

7. Create the .env file

```
GROQ_API_KEY=PASTE_YOUR_GROQ_API_KEY_HERE
GROQ_MODEL=openai/gpt-oss-20b
```

The application reads GROQ_API_KEY and defaults GROQ_MODEL to openai/gpt-oss-20b if no model is specified. ■filecite■turn0file0■L17-L31■

8. Run the application

```
python app.py
```

Expected terminal output will contain a Flask development-server message and a local address similar to http://127.0.0.1:5000/. Open that address in your browser. The code runs Flask on host 127.0.0.1 and port 5000 with debug=True. ■filecite■turn0file0■L569-L577■

Chat Scholar — Execution & Testing

9. Test PDF Chat — step by step

1) Open the local Flask URL. 2) Go to the PDF upload/chat page from your application's interface. 3) Select one or more readable PDF files. 4) Submit/process them. 5) Ask a question about the uploaded PDF. The processing route extracts PDF text, splits it into chunks, creates the FAISS vector store and redirects to chat. ■filecite■turn0file0■L395-L425■

The default chunking is 1000 characters with 200-character overlap. FAISS searches up to 4 chunks for a question. ■filecite■turn0file0■L159-L184■ ■filecite■turn0file0■L212-L240■

10. Example terminal test sequence

```
(.venv) PS C:\...\ChatScholar> python app.py
 * Running on http://127.0.0.1:5000
 * Debugger is active

Browser:
http://127.0.0.1:5000/

Upload: sample.pdf
Question: What is the main topic of this document?
Result: answer generated from retrieved PDF context.
```

11. Test Essay Grading

Open the essay-grading page. Paste an essay or upload a readable PDF. The application extracts the PDF text when a file is supplied, otherwise it reads the essay text from the form, then sends it to Groq using the configured rubric. ■filecite■turn0file0■L485-L542■

12. Important files/folders

```
ChatScholar/
■  app.py
■  .env
■  ■■■ templates/
■       ■■■ new_home.html
■       ■■■ new_chat.html
■       ■■■ new_pdf_chat.html
■       ■■■ new_essay_grading.html
■       ■■■ new_essay_rubric.html
■■■ __data__/
```

Your Python code calls render_template() for these HTML templates, so those template files must exist in a templates folder. It also creates __data__ automatically for extracted text. ■filecite■turn0file0■L34-L43■ ■filecite■turn0file0■L387-L392■

13. If you get errors

```
GROQ_API_KEY is missing
→ Check .env and make sure the key name is exactly GROQ_API_KEY.

ModuleNotFoundError
→ Activate .venv, then run:
python -m pip install <missing-package>

PDF processing failed
→ Check the Terminal traceback. The app rejects encrypted PDFs and may not extract text from scanned/image-only PDFs.

Chat failed
→ Check GROQ_API_KEY, GROQ_MODEL and the Terminal traceback.
```

14. Stop and restart

```
Stop server:
Ctrl + C

Start again:
.\.venv\Scripts\Activate.ps1
python app.py
```

15. Quick checklist

- ■ Python 3.12 environment activated
- ■ app.py is in the project folder
- ■ templates folder exists with required HTML files
- ■ .env contains GROQ_API_KEY
- ■ GROQ_MODEL is a currently available Groq model
- ■ Required packages installed
- ■ python app.py runs without traceback
- ■ Browser opens http://127.0.0.1:5000/
- ■ PDF uploads successfully
- ■ PDF question returns an answer
- ■ Essay grading works

The uploaded code intentionally avoids PyTorch/HuggingFace/OpenAI embeddings and uses its own lightweight hashing-based embedding function. ■filecite■turn0file0■L71-L106■